

ANN and DL (CAI-361)

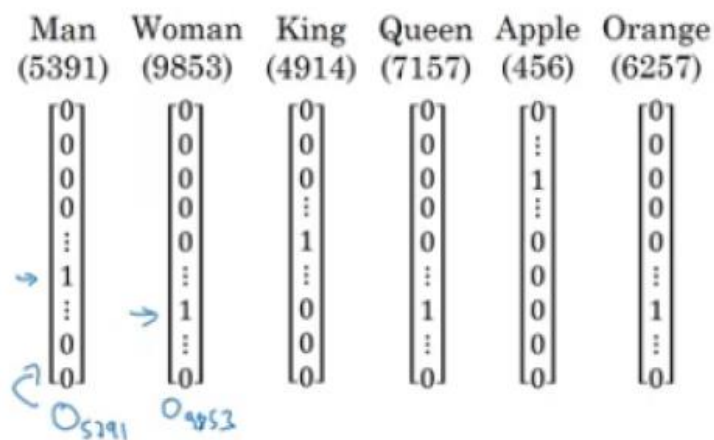
Objectives:

- Embeddings
- Transformers

Embeddings:

Embeddings are numerical representations of objects, such as words or images, that capture their meanings or characteristics in a way that makes it easy for machines to process and analyse them. These representations are typically in the form of vectors.

Word embeddings is a way of representing words. It lets your algorithm automatically understand the analogies between words like "king" and "queen". So far you have learnt defining our language by a vocabulary. Then representing them with a **onehot** vector that represents the word in the vocabulary. We will use the annotation O_{idx} for any word that is represented with one-hot like in the image.



One of the weaknesses of this representation is that it treats a word as a thing that itself and it doesn't allow an algorithm to generalize across words.

For example: "I want a glass of orange _____", a model should predict the next word as juice.

A similar example "I want a glass of apple _____", a model won't easily predict juice here if it wasn't trained on it. And if so the two examples aren't related although orange and apple are similar. Inner product between any one-hot encoding vector is zero. Also, the distances between them are the same. So, instead of a **one-hot presentation**, it would be better to learn a **featurized representation** with each of these words: man, woman, king, queen, apple, and orange.

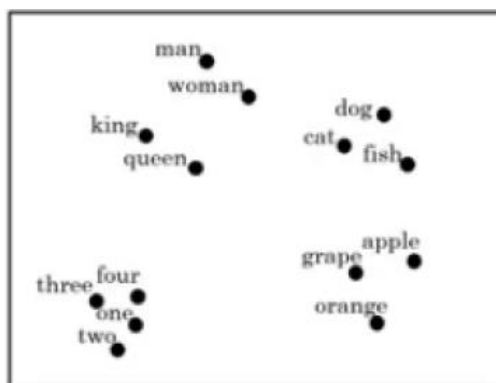
	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	<u>0.93</u>	<u>0.95</u>	-0.01	0.00
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.04	0.01	0.02	0.01	0.95	0.97
size						
cost						
alike						
verb						

- Each word will have a, for example, 300 features with a type of float point number.
- Each word column will be a 300-dimensional vector which will be the representation.
- We will use the notation $\mathbf{e}_{539}$ to describe man word features vector.
- Now, if we return to the examples we described again:

"I want a glass of orange _____"

I want a glass of apple _____

- Orange and apple now share a lot of similar features which makes it easier for an algorithm to generalize between them.
- We call this representation Word embeddings.
- To visualize word embeddings we use a t-SNE algorithm to reduce the features to 2 dimensions which makes it easy to visualize:



You will get a sense that more related words are closer to each other. The word embeddings came from that we need to embed a unique vector inside a ndimensional space.

One of the most fascinating properties of word embeddings is that they can also help with **analogy reasoning**. While analogy reasoning may not be by itself the most important NLP application, but it might help convey a sense of what these word embeddings can do.

Analogies example:

Given this word embeddings table:

	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0.00
Age	0.03	0.02	0.70	0.69	0.03	-0.02
Food	0.09	0.01	0.02	0.01	0.95	0.97
	e_{Man}	e_{Woman}	e_{King}	e_{Queen}		

Can we conclude this relation:

Man ==> Woman

King ==> ??

Lets subtract e_{Man} from e_{Woman} . This will equal the vector [-2 0 0 0]

Similar $e_{\text{King}} - e_{\text{Queen}} = [-2 0 0 0]$

So we can reformulate the problem to find:

$$e_{\text{Man}} - e_{\text{Woman}} \approx e_{\text{King}} - e_{\text{Queen}}$$

It turns out that e_{Queen} is the best solution here that gets the the similar vector. The most commonly used similarity function is called **Cosine similarity**.

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

You can also use Euclidean distance as a similarity function (but it rather measures a dissimilarity, so you should take it with negative sign).

We can use this equation to calculate the similarities between word embeddings and on the analogy problem where $\mathbf{u} = e_{\text{Woman}} - e_{\text{Man}}$ and $\mathbf{v} = e_{\text{Queen}} - e_{\text{King}}$

When you implement an algorithm to learn a word embedding, what you end up learning is a embedding matrix.

What are transformers?

LSTMs and RNNs present 3 problems:

- Sequential computation inhibits parallelization
- No explicit modeling of long and short range dependencies
- “Distance” between positions is linear

Transformers were invented to address several key limitations of Recurrent Neural Networks (RNNs) and Long Short-Term Memory networks (LSTMs). Here are the basic reasons for the development of transformers

- Parallelization
- Attention mechanism
- Handling Long-Range Dependencies
- Efficiency in Learning Representations

Transformers are a type of neural network just like recurrent neural networks (RNNs) or convolutional neural networks (CNNs).

There are 3 key elements that make transformers so powerful:

- Self-attention
- Positional embeddings
- Multihead attention

All of them were introduced in 2017 in the “**Attention Is All You Need**” paper by Vaswani et al. In that paper, authors proposed a completely new way of approaching deep learning tasks such as machine translation, text generation, and sentiment analysis.

The self-attention mechanism enables the model to detect the connection between different elements even if they are far from each other and assess the importance of those connections, therefore, improving the understanding of the context. According to Vaswani, “Meaning is a result of relationships between things, and self-attention is a general way of learning relationships.”

Due to positional embeddings and multihead attention, transformers allow for simultaneous sequence processing, which means that model training can be sped up through parallelization. This is a huge benefit of using transformers over architectures like RNN and has enabled the creation of large language models. For example, BERT (Bidirectional Encoder Representations from Transformers) and GPT (Generative Pretrained Transformer) are, as you might have guessed, variants of the original transformer architecture. These models have achieved state-of-the-art performance on a wide range of NLP tasks and have helped to establish the transformer as the dominant architecture for NLP. As they evolve, we’ll see even more exciting advances in the years to come.

How do transformers work?

The key to understanding why transformers are used much more frequently than other neural networks is to look at what lies inside them.

Word embeddings

Words are awkward to use in a machine learning model, so they need to be transformed into something that a machine learning model can operate with – vectors. This is done with the help of word embeddings, which are vector representations of words. All the words in the English

language can, it turns out, be assigned a vector value that encodes their meaning – the meaning being their relationships with other words in the English language.

The most famous algorithm for doing this is **Word2vec**. It takes a corpus of data and learns word embeddings in an unsupervised manner from the raw data.

If you would like to learn more in detail about word vectors, we recommend this great article by Adrian Colyer.

Positional embeddings

In NLP, sequences (like sentences or documents) of text are typically represented as vectors of word embeddings. However, a word embedding doesn't capture the position of the word in the sequence. This is where positional embeddings come into play: they encode the position of each token in a sequence, and so add this positional information to the word embeddings. By incorporating positional embeddings into the input representation of an NLP model, the model is able to capture the order and position of words in a sequence. This is important for a model like transformers where words are not processed in sequence.

Encoder/decoder

Transformers use a seq2seq neural network architecture that transforms an input sequence of vectors into an output sequence by passing it through a series of encoder and decoder layers.

The goal of each encoder layer is to extract features from a sequence, while the goal of each decoder layer is to use the features to produce an output sequence. In transformers, encoding and decoding is done by using attention.

Attention

Attention helps the model to understand the context of a word by considering words that go before and after it.

For example, the word “bark” can be connected to dogs but it can also be “tree bark”. The context of other words in the sentence helps computer to understand different meanings of the word and translate accordingly:

“I could hear the dog bark”. Somewhere next to the word bark in the vocabulary you would find words “dog”, “loudly”, “car”.

“The tree bark was wet after the storm”. Somewhere next to this representation you would find the words “brown”, “rough”, “tree”, “park”.

Self-attention

Attention and self-attention are both important to the transformer architecture and shouldn't be used interchangeably. Attention mechanism helps the model to analyze different parts of another

sequence, which is the sequence that is being generated by the decoder. Self-attention mechanism works with the sequence that is being encoded. Self-attention mechanism allows the model to weigh the importance of different parts of the input sequence against each other. This is done by computing a set of attention weights that indicate the relevance of each element in the input sequence to every other element. By doing this, the model is able to effectively capture long-range dependencies in the input sequence and learn to recognize patterns that span multiple elements.

Multi-head attention

Multi-head attention allows the network to learn multiple ways of weighing the input sequence against itself. The vectors responsible for tokens are broken up into multiple parts called heads (8 in the original paper) which go through the same attention computing process as before. The results of the process are concatenated together to form an output of the same type. This process can be parallelized, which enables training the model faster. It also allows the model to learn the context of the words better.

